

TypeIT : Real-Time, Multi-Client chat App with AI integration at its heart

Advanced Programming Final Project Report

University	E-JUST
Course	Advanced Programming
Student Name	Mahmoud Mostafa
ID	320250263
Section	10

Problem Solved

Normal text chat and communication apps disconnect Human interactions and machine (AI) from each other, File sharing mostly depends on cloud solutions instead of a native solution integrated to peer to peer pipeline which increases latency, dependencies, and security risks

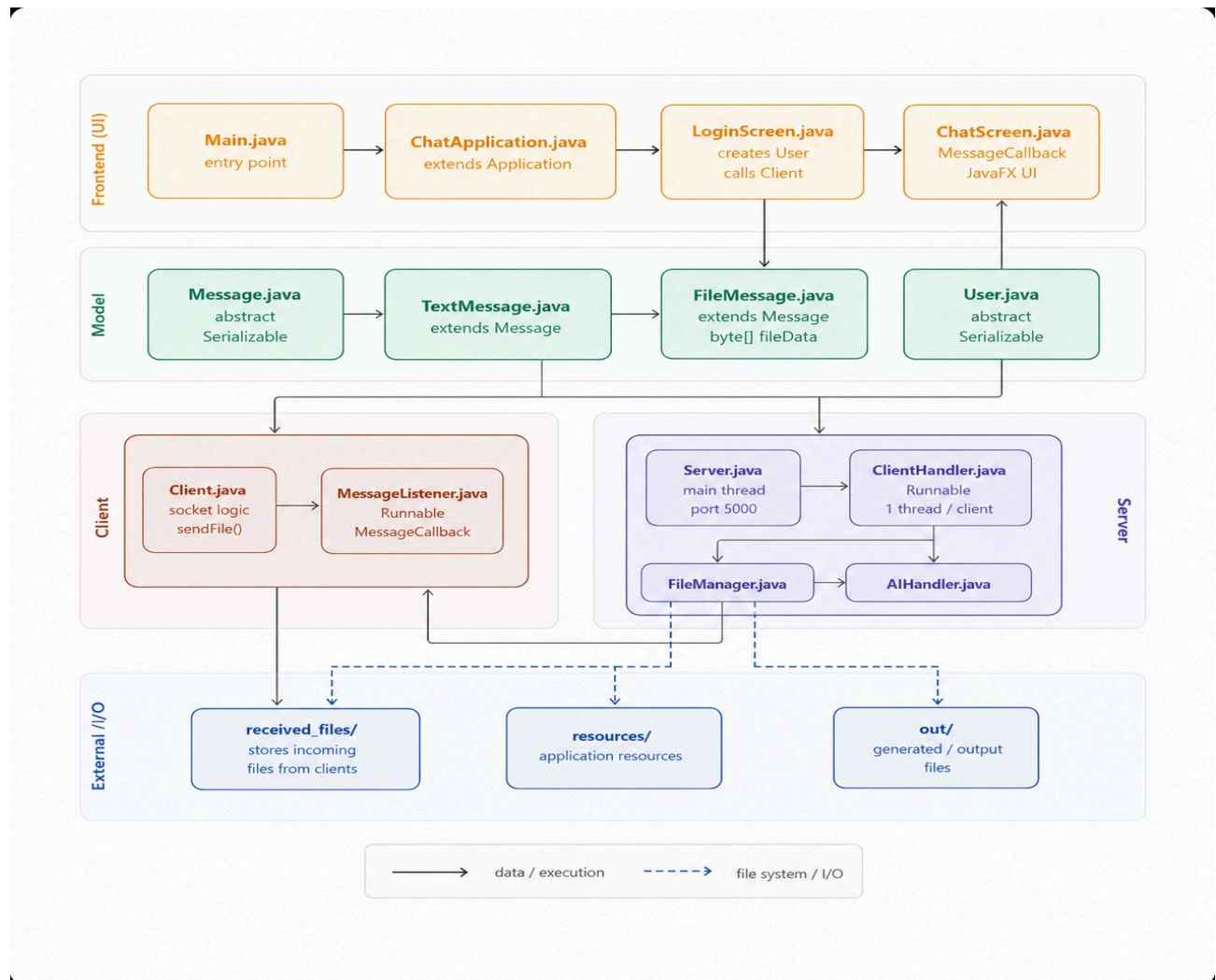
TypeIT changes this by implementing a unified, multi-threaded, localized communications platform, its feature enable real time text messages, binary file & image sync across a byte stream pipeline, and instant access to LLMs within the app by simply invoking it using the command prefix (/ai)

Workflow Sequence

1. **Initialization:** The Main bootstrap class triggers the JavaFX lifecycle via `ChatApplication`, which builds the primary Window state (Stage) and paints the initial `LoginScreen` view
2. **Handshake and Registration:** The user inputs a profile establishes a privilege role selection (`AdminUser` vs `RegularUser`). The underlying Client instance initializes a persistent Socket channel targeted at the host server on designated `Port 5000`
3. **Server Isolation and Thread Pooling:** The background listening loop inside the Server class intercepts the incoming connection request. It registers the active socket instance to a global thread-safe and passes ownership of the execution lifecycle over to an isolated, runnable `ClientHandler` background thread
4. **Asynchronous Frontend Listening:** Concurrently, the frontend application layer launches a background instance via `MessageListener` This thread blocks on stream-read

operations to continuously capture incoming server broadcasts without freezing t

System Architecture



Networking & Multi-Threading

A Thread-Per-Client Concurrency Pattern

To not process blockages across a multi-user environment, the system handles nodes inside scheduling routines. When a client starts a heavy transaction such as uploading a dense binary file or waiting for an external AI text generation handshake over the internet the operations take place completely inside an isolated execution thread. This pattern helps prevent any blocking of

the main network loop or causing latency & delays for other users

OOP Implementation

Concept	Implementation
Encapsulation	Data restricted using private or private final modifiers. Changing it is controlled through constructor parameters and explicit accessor methods
Inheritance	Extensive code reusability via class hierarchies. Networks derive from an abstract base class (<code>TextMessage</code> and <code>FileMessage</code> extend <code>Message</code>), and users derived from an abstract <code>User</code> class (<code>RegularUser</code> and <code>AdminUser</code>)
Polymorphism	Dynamic overriding across streams, The base <code>Message</code> class defines an abstract <code>format()</code> method signature, which gets overridden by <code>TextMessage</code> and <code>FileMessage</code> to serialize output formatting for the server logs. Run-time pattern matching handles class switching dynamically
Abstraction	The core data entities inherit requirements from <code>java.io.Serializable</code> , asynchronous threads execution through <code>java.lang.Runnable</code> implementations, and the UI separated from the listener a custom callback interface (<code>MessageCallback</code>)

File Handling & Data Security Protocols

High-Performance Object Serialization

Instead of segmenting text and files into fragmented character streams, the platform uses high-level `ObjectOutputStream` and `ObjectInputStream` pipelines. During file transmissions, the `FileManager` extracts raw byte streams (`byte[]`) from the disk. The raw file data is wrapped into a `FileMessage` object and serialized across the network, the raw byte array gets read and reconstructed through extension formatting, and writes a binary duplicate into the server storage folder (`received_files/`)

API Credentials & Code Safety

To follow security principles, the server prevents confidential authentication parameters such as Cloud AI provider keys) from being hardcoded inside plain text files. The application uses system environment configurations through `System.getenv("GROQ_API_KEY")`, And an `.env` file which is locally implemented to pass variables to the compiler, while an optimized `.gitignore` rule isolates the file from version control which eliminates the risk of leaks when pushed to public repositories

AI Assistant Architecture

The AI module is tracked and managed within the `AIHandler` component. This provides that any message containing `/ai` keyword is identified to process an API query request

- The system clears the command to and keeps only the query text
- A thread works in the background to only handle the web
- The query has mutli-line text (`"""`) to format a `JSON` into a structured request body
- The request is sent to Groq server through Java's `httpClient` secured with the Token
- A successful request responds with 200 OK code, then a text parser finds the `string content` field, ignores the `JSON` formatting and broadcasts the message to the chat under the name  **AI Assistant**.

Libraries and Dependencies

1. User Interface

- JavaFX with `Javafx.controls`, `javafx.fxml`
- Used to design and render the GUI

1. Networking and Concurrency

- Java Sockets with [java.net.Socket](#), [java.net.ServerSocket](#) and [java.lang.Thread](#)
- handles stream pipelines. The ServerSocket establishes a dedicated port listener to accept incoming requests, spawning a multi-threaded '[ClientHandler](#)' thread for every connected user to handle parallel data processing

1. AI Engine Integration

- Java Http client ([java.net.http.Httpclient](#))
- Manages the communications between the chat app and the cloud LLM handshakes